

Ansible — Panduan Belajar Step-by-Step

Target: Pemula DevOps — tanpa asumsi pengalaman Ansible

Prasyarat: Terminal Linux/macOS/WSL, Python 3, SSH client

Daftar Isi

- Apa Itu Ansible?
 - Instalasi
 - Inventory — Daftar Server Target
 - Ad-hoc Commands — Eksekusi Cepat
 - Playbook — Automation Blueprint
 - Variables — Data Dinamis
 - Conditionals & Loops
 - Roles — Modular Automation
 - Ansible Vault — Enkripsi Secrets
 - Real-World Projects
 - Cheatsheet & Referensi Cepat
-

1. Apa Itu Ansible?

Ansible adalah automation engine open-source buatan Red Hat. Ia melakukan provisioning, configuration management, application deployment, dan orchestration — semuanya tanpa agent di server target.

Kenapa Ansible?

| Fitur | Penjelasan |

|---|---|

| Agentless | Tidak perlu install agent di server target. Hanya perlu SSH + Python. |

| Idempotent | Menjalankan task yang sama berkali-kali = hasil yang sama. Aman. |

| YAML-based | Playbook ditulis dalam YAML — mudah dibaca manusia. |

| Push-based | Control node PUSH konfigurasi ke managed nodes. |

| Module-rich | Ribuan modul built-in: package, file, service, cloud, database, dll. |

Arsitektur

```
Control Node (laptop/VM) ---SSH--> Managed Node 1 (server)
ansible + playbook.yml ---SSH--> Managed Node 2 (server)
inventory.ini           ---SSH--> Managed Node 3 (server)
```

- Control Node: Mesin tempat Ansible diinstall dan playbook dijalankan.
- Managed Nodes: Server-server yang dikelola. Tidak perlu install apapun.
- Inventory: File berisi daftar managed nodes.
- Modules: Unit kerja Ansible (contoh: copy, apt, service, user).
- Playbook: File YAML berisi rangkaian task.

Module vs Playbook vs Role

- Module = Satu fungsi atomik (apt, copy, service, user, file)
 - Playbook = Rangkaian task yang memanggil module
 - Role = Playbook yang diorganisir menjadi komponen reusable
-

2. Instalasi

2.1 Di macOS

```
# Install Homebrew dulu jika belum
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install Ansible
brew install ansible

# Verifikasi
ansible --version
```

2.2 Di Ubuntu/Debian

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible

ansible --version
```

2.3 Via pip (semua OS)

```
python3 -m pip install --user ansible

# Pastikan ~/.local/bin ada di PATH
export PATH="$HOME/.local/bin:$PATH"
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc

ansible --version
```

2.4 Setup Awal — Buat Project Lab

```
mkdir -p ~/ansible-lab
cd ~/ansible-lab
```

3. Inventory — Daftar Server Target

Inventory adalah file yang mendefinisikan managed nodes. Format: INI atau YAML.

3.1 Inventory INI Dasar

Buat file inventory.ini:

```
# inventory.ini

# Server tunggal
web1 ansible_host=192.168.1.10 ansible_user=ubuntu

# Group servers
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20

# Group of groups
[production:children]
webservers
dbservers

# Variables per-group
[webservers:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[dbservers:vars]
ansible_user=admin
ansible_port=2222
```

3.2 Inventory YAML (Rekomendasi)

Buat file inventory.yml:

```
# inventory.yml
all:
  hosts:
    local-vm:
      ansible_host: 127.0.0.1
      ansible_connection: local          # lokal -> tanpa SSH
  children:
    webservers:
      hosts:
        web1:
          ansible_host: 192.168.1.10
        web2:
          ansible_host: 192.168.1.11
      vars:
        ansible_user: ubuntu
    dbservers:
      hosts:
        db1:
          ansible_host: 192.168.1.20
      vars:
        ansible_user: admin
```

3.3 Cek Inventory

```
# List semua hosts
ansible-inventory -i inventory.yml --list

# Graph tree
ansible-inventory -i inventory.yml --graph
```

3.4 Latihan: Local-only Inventory

Untuk belajar tanpa server remote, gunakan localhost:

```
# inventory-local.yml
all:
  hosts:
    localhost:
      ansible_connection: local
```

4. Ad-hoc Commands — Eksekusi Cepat

Ad-hoc commands menjalankan satu module tanpa playbook. Cocok untuk operasi cepat atau testing.

Sintaks Dasar

```
ansible <pattern> -i <inventory> -m <module> -a "<arguments>"
```

4.1 Ping Semua Host

```
ansible all -i inventory-local.yml -m ping
```

Output yang diharapkan:

```
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

4.2 Module command — Jalankan Shell Command

```
# Cek hostname
ansible all -i inventory-local.yml -m command -a "hostname"

# Cek uptime
ansible all -i inventory-local.yml -m command -a "uptime"

# Cek disk usage
ansible all -i inventory-local.yml -m command -a "df -h"
```

Catatan: Module command tidak memproses shell pipes/redirects. Untuk itu gunakan module shell.

4.3 Module shell — Command dengan Shell

```
# Menggunakan pipe
ansible all -i inventory-local.yml -m shell -a "ps aux | grep python"

# Menggunakan redirect
ansible all -i inventory-local.yml -m shell -a "echo 'hello' > /tmp/test.txt"
```

4.4 Module copy — Copy File

```
ansible all -i inventory-local.yml -m copy -a "src=/etc/hosts dest=/tmp/hosts-backup"
```

4.5 Module file — Manage File & Directory

```
# Buat directory
ansible all -i inventory-local.yml -m file -a "path=/tmp/mydir state=directory mode=0755"

# Buat file kosong (touch)
ansible all -i inventory-local.yml -m file -a "path=/tmp/test.txt state=touch"

# Hapus file/directory
ansible all -i inventory-local.yml -m file -a "path=/tmp/test.txt state=absent"
```

4.6 Module apt / yum — Package Management

```
# Ubuntu/Debian: install nginx
ansible webserver -i inventory.ini -m apt -a "name=nginx state=present" -b

# CentOS/RHEL: install nginx
ansible webserver -i inventory.ini -m yum -a "name=nginx state=present" -b
```

Flag `-b` = `--become` = `sudo`.

4.7 Module service — Manage Service

```
# Start nginx
ansible webserver -i inventory.ini -m service -a "name=nginx state=started" -b

# Restart nginx
ansible webserver -i inventory.ini -m service -a "name=nginx state=restarted" -b

# Enable on boot
ansible webserver -i inventory.ini -m service -a "name=nginx enabled=yes" -b
```

4.8 Module setup — Gather Facts

```
# Lihat SEMUA fakta dari host
ansible all -i inventory-local.yml -m setup

# Filter fakta spesifik
ansible all -i inventory-local.yml -m setup -a "filter=ansible_distribution"
ansible all -i inventory-local.yml -m setup -a "filter=ansible_memtotal_mb"
ansible all -i inventory-local.yml -m setup -a "filter=ansible_processor"
```

4.9 Module user — Manage User

```
# Buat user
ansible all -i inventory-local.yml -m user -a "name=testuser state=present shell=/bin/bash" -b

# Hapus user
ansible all -i inventory-local.yml -m user -a "name=testuser state=absent remove=yes" -b
```

4.10 Module get_url — Download File

```
ansible all -i inventory-local.yml -m get_url -a "url=https://example.com/file.tar.gz dest=/tmp/file..."
```

4.11 Latihan Mandiri

Jalankan command berikut dan catat outputnya:

```
# 1. Cek OS dan versi
ansible all -i inventory-local.yml -m setup -a "filter=ansible_distribution*"

# 2. Cek total memory
ansible all -i inventory-local.yml -m setup -a "filter=ansible_memtotal_mb"

# 3. Buat file di /tmp
ansible all -i inventory-local.yml -m file -a "path=/tmp/ansible-test state=touch"

# 4. Cek file ada
ansible all -i inventory-local.yml -m command -a "ls -la /tmp/ansible-test"

# 5. Hapus file
ansible all -i inventory-local.yml -m file -a "path=/tmp/ansible-test state=absent"
```

5. Playbook — Automation Blueprint

Playbook adalah file YAML berisi daftar plays (apa yang dikerjakan, di host mana).

5.1 Playbook Pertama

Buat file first-playbook.yml:

```
---
# first-playbook.yml
- name: Setup Web Server
  hosts: all
  become: yes

  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Start Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
```

Jalankan:

```
ansible-playbook -i inventory.ini first-playbook.yml
```


5.2 Cek Sintaks Sebelum Run

```
# Syntax check (dry-run parsing)
ansible-playbook -i inventory.ini first-playbook.yml --syntax-check

# Dry-run (simulasi, tidak benar-benar mengubah)
ansible-playbook -i inventory.ini first-playbook.yml --check

# Dry-run + diff (lihat apa yang akan berubah)
ansible-playbook -i inventory.ini first-playbook.yml --check --diff

# Step-by-step (konfirmasi per task)
ansible-playbook -i inventory.ini first-playbook.yml --step
```

5.3 Playbook Lengkap — Setup Web Server

Buat file `setup-webserver.yml`:

```

---
# setup-webserver.yml
- name: Setup Web Server
  hosts: all
  become: yes

  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: yes
        cache_valid_time: 3600
        when: ansible_os_family == "Debian"

    - name: Install required packages
      ansible.builtin.apt:
        name:
          - nginx
          - git
          - curl
          - htop
        state: present
        when: ansible_os_family == "Debian"

    - name: Ensure Nginx is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes

    - name: Create web root
      ansible.builtin.file:
        path: /var/www/mysite
        state: directory
        owner: www-data
        group: www-data
        mode: '0755'

    - name: Copy custom index.html
      ansible.builtin.copy:
        content: |
          <!DOCTYPE html>
          <html>
          <head><title>Deployed by Ansible</title></head>
          <body>
            <h1>Hello from Ansible!</h1>
            <p>Server: {{ ansible_hostname }}</p>
          </body>
          </html>
        dest: /var/www/mysite/index.html
        owner: www-data
        group: www-data
        mode: '0644'

    - name: Configure Nginx site
      ansible.builtin.copy:
        content: |
          server {
            listen 80;
            server_name {{ ansible_hostname }};
            root /var/www/mysite;
            index index.html;
            location / {
              try_files $uri $uri/ =404;

```

```

    }
  }
  dest: /etc/nginx/sites-available/mysite
  notify: Reload Nginx

- name: Enable site
  ansible.builtin.file:
    src: /etc/nginx/sites-available/mysite
    dest: /etc/nginx/sites-enabled/mysite
    state: link
  notify: Reload Nginx

- name: Remove default site
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Reload Nginx

handlers:
- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded

```

5.4 Memahami Handlers

Handlers adalah task yang HANYA dijalankan jika di-trigger oleh notify.

- Jika TIDAK ADA perubahan di task yang me-notify -> handler TIDAK dijalankan
- Jika ADA perubahan -> handler dijalankan sekali saja di akhir play

5.5 Task Control Flags

```

- name: Contoh task dengan semua opsi kontrol
  ansible.builtin.command: /usr/bin/some-command
  ignore_errors: yes          # Lanjut meskipun error
  changed_when: false        # Jangan tandai sebagai "changed"
  failed_when:                # Custom failure condition
    - result.rc != 0
    - "'FATAL' in result.stderr"
  register: my_result         # Simpan output ke variable
  tags:
    - setup
    - critical

```

Menjalankan dengan tag:

```

ansible-playbook playbook.yml --tags setup
ansible-playbook playbook.yml --skip-tags critical

```

6. Variables — Data Dinamis

6.1 Variable di Playbook

```
---
- name: Demo Variables
  hosts: all
  vars:
    app_name: "myapp"
    app_port: 8080
  tasks:
    - name: Print variables
      ansible.builtin.debug:
        msg: "App: {{ app_name }}, Port: {{ app_port }}"
```

6.2 Variable dari File Eksternal (vars_files)

Buat vars/app.yml:

```
# vars/app.yml
app_name: "myapp"
app_version: "1.2.3"
app_port: 8080
```

Gunakan di playbook:

```
- name: Load vars from file
  hosts: all
  vars_files:
    - vars/app.yml
  tasks:
    - ansible.builtin.debug:
        msg: "Deploying {{ app_name }} v{{ app_version }} on port {{ app_port }}"
```

6.3 Group & Host Variables (Best Practice)

Struktur direktori profesional:

```

project/
  ansible.cfg
  inventory.yml
  group_vars/
    all.yml           # Variables untuk semua hosts
    webservers.yml    # Variables untuk group webservers
    dbservers.yml     # Variables untuk group dbservers
  host_vars/
    web1.yml          # Variables spesifik web1
    db1.yml           # Variables spesifik db1
  playbook.yml

```

Contoh group_vars/webservers.yml:

```

nginx_port: 80
nginx_user: www-data
app_root: /var/www/myapp

```

6.4 Variable Precedence (dari rendah ke tinggi)

- Role defaults
- Inventory group_vars/all
- Playbook group_vars/all
- Inventory group_vars/*
- Playbook group_vars/*
- Inventory host_vars/*
- Host facts / cached facts
- Play vars
- Play vars_prompt / vars_files
- Role vars (vars/main.yml)
- Block vars / Task vars
- set_fact / registered vars
- Extra vars (-e "key=value") — TERTINGGI

6.5 Fakta (Facts) — Variable Bawaan

```
- name: Demo Facts
hosts: all
tasks:
  - name: Tampilkan fakta penting
    ansible.builtin.debug:
      msg:
        - "Hostname: {{ ansible_hostname }}"
        - "OS: {{ ansible_distribution }} {{ ansible_distribution_version }}"
        - "Arch: {{ ansible_architecture }}"
        - "CPU cores: {{ ansible_processor_cores }}"
        - "RAM (MB): {{ ansible_memtotal_mb }}"
        - "IPv4: {{ ansible_default_ipv4.address }}"
```

Nonaktifkan fact gathering jika tidak diperlukan:

```
- name: Fast playbook
hosts: all
gather_facts: no
```

7. Conditionals & Loops

7.1 when — Conditional Execution

```

---
- name: Demo Conditionals
  hosts: all
  vars:
    deploy_ssl: true
  tasks:
    - name: Install Nginx hanya di Debian
      ansible.builtin.apt:
        name: nginx
        state: present
        when: ansible_os_family == "Debian"

    - name: Copy SSL cert hanya jika deploy_ssl=true
      ansible.builtin.copy:
        src: ssl/cert.pem
        dest: /etc/nginx/ssl/cert.pem
        when: deploy_ssl | bool

    - name: Restart hanya di production + Debian (AND)
      ansible.builtin.service:
        name: nginx
        state: restarted
        when:
          - ansible_os_family == "Debian"
          - inventory_hostname in groups['production']

    - name: Cek apakah file config ada
      ansible.builtin.stat:
        path: /etc/myapp/config.yml
        register: config_file

    - name: Backup config jika ada
      ansible.builtin.copy:
        src: /etc/myapp/config.yml
        dest: /etc/myapp/config.yml.bak
        when: config_file.stat.exists

```

7.2 loop — Iterasi

```

---
- name: Demo Loops
  hosts: all
  tasks:
    # Loop sederhana — install multiple packages
    - name: Install packages
      ansible.builtin.apt:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
        - git
        - curl
        - htop

    # Loop dengan dictionary
    - name: Create multiple users
      ansible.builtin.user:
        name: "{{ item.name }}"
        group: "{{ item.group }}"
        shell: "{{ item.shell }}"
        state: present
      loop:
        - { name: "alice", group: "developers", shell: "/bin/bash" }
        - { name: "bob",   group: "developers", shell: "/bin/bash" }
        - { name: "carol", group: "admins",     shell: "/bin/zsh" }

```

7.3 block — Grouping Tasks + Error Handling

```

---
- name: Demo Block
  hosts: all
  tasks:
    - name: Critical operation with rescue
      block:
        - name: Attempt risky operation
          ansible.builtin.command: /usr/bin/risky-command
        - name: Runs only if risky-command succeeds
          ansible.builtin.debug:
            msg: "Operation successful"
      rescue:
        - name: Rollback jika gagal
          ansible.builtin.debug:
            msg: "ROLLING BACK!"
      always:
        - name: Selalu dijalankan (cleanup)
          ansible.builtin.file:
            path: /tmp/lockfile
            state: absent

```

8. Roles — Modular Automation

Role adalah cara mengorganisir playbook menjadi komponen reusable.

8.1 Struktur Role

```
roles/
  webserver/
    tasks/          # Task utama (main.yml)
    handlers/       # Handlers (main.yml)
    vars/           # Variables HIGH precedence (main.yml)
    defaults/       # Default variables LOW precedence (main.yml)
    templates/      # Jinja2 templates (*.j2)
    files/          # Static files
    meta/           # Role metadata & dependencies (main.yml)
```

8.2 Buat Role

```
cd ~/ansible-lab

# Buat role otomatis
ansible-galaxy init roles/webserver

# Atau buat manual:
mkdir -p roles/webserver/{tasks,handlers,vars,defaults,templates,files,meta}
```

8.3 Isi Role — roles/webserver/tasks/main.yml

```

---
- name: Update apt cache
  ansible.builtin.apt:
    update_cache: yes
    cache_valid_time: 3600
  when: ansible_os_family == "Debian"

- name: Install Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
  when: ansible_os_family == "Debian"

- name: Create web root directory
  ansible.builtin.file:
    path: "{{ web_root }}"
    state: directory
    owner: "{{ web_user }}"
    group: "{{ web_user }}"
    mode: '0755'

- name: Deploy index.html from template
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ web_root }}/index.html"
    owner: "{{ web_user }}"
    group: "{{ web_user }}"
    mode: '0644'

- name: Deploy Nginx config from template
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/sites-available/{{ site_name }}
  notify: Reload Nginx

- name: Enable site
  ansible.builtin.file:
    src: /etc/nginx/sites-available/{{ site_name }}
    dest: /etc/nginx/sites-enabled/{{ site_name }}
    state: link
  notify: Reload Nginx

- name: Remove default site
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Reload Nginx

- name: Start & enable Nginx
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes

```

8.4 Default Variables — roles/webserver/defaults/main.yml

```
---
web_root: /var/www/mysite
web_user: www-data
site_name: mysite
nginx_port: 80
app_name: My Application
```

8.5 Handlers — roles/webserver/handlers/main.yml

```
---
- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

8.6 Jinja2 Templates

roles/webserver/templates/index.html.j2:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ app_name }}</title>
  <style>
    body { font-family: sans-serif; max-width: 800px; margin: 50px auto; }
    .info { background: #f0f0f0; padding: 20px; border-radius: 8px; }
  </style>
</head>
<body>
  <h1>{{ app_name }}</h1>
  <div class="info">
    <p><strong>Hostname:</strong> {{ ansible_hostname }}</p>
    <p><strong>OS:</strong> {{ ansible_distribution }} {{ ansible_distribution_version }}</p>
    <p><strong>Deployed by:</strong> Ansible</p>
    <p><strong>Date:</strong> {{ ansible_date_time.date }}</p>
  </div>
</body>
</html>
```

roles/webserver/templates/nginx.conf.j2:

```
server {
    listen {{ nginx_port }};
    server_name {{ ansible_hostname }};
    root {{ web_root }};
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/{{ site_name }}_access.log;
    error_log /var/log/nginx/{{ site_name }}_error.log;
}
```

8.7 Gunakan Role di Playbook

Buat site.yml:

```
---
- name: Deploy Web Application
  hosts: all
  become: yes
  roles:
    - webserver
```

Jalankan:

```
ansible-playbook -i inventory.ini site.yml
```

9. Ansible Vault — Enkripsi Secrets

JANGAN PERNAH commit password, API keys, atau secrets ke git. Gunakan Vault.

9.1 Enkripsi File

```
# Buat file terenkripsi
ansible-vault create secrets.yml

# Enkripsi file yang sudah ada
ansible-vault encrypt vars/production.yml

# Edit file terenkripsi
ansible-vault edit secrets.yml

# Lihat isi
ansible-vault view secrets.yml
```

9.2 Gunakan di Playbook

```
---
- name: Deploy with secrets
  hosts: all
  vars_files:
    - secrets.yml
  tasks:
    - name: Use secret in template
      ansible.builtin.template:
        src: app.conf.j2
        dest: /etc/myapp/app.conf
```

9.3 Jalankan Playbook dengan Vault

```
# Prompt password
ansible-playbook -i inventory.ini site.yml --ask-vault-pass

# Dari file password
echo "my-vault-password" > .vault_pass
chmod 600 .vault_pass
ansible-playbook -i inventory.ini site.yml --vault-password-file .vault_pass

# Dari environment variable
export ANSIBLE_VAULT_PASSWORD_FILE=.vault_pass
ansible-playbook -i inventory.ini site.yml
```

9.4 Enkripsi Variable Tunggal (Best Practice)

```
ansible-vault encrypt_string "my-secret-password" --name "db_password"
```

Output:

```
db_password: !vault |
               $ANSIBLE_VAULT;1.1;AES256
               66386439653236336...
```

Copy-paste ke file `group_vars/all.yml`.

10. Real-World Projects

10.1 Project 1: Nginx + Static Website

Struktur project:

```
project-website/
  ansible.cfg
  inventory.yml
  site.yml
  group_vars/
    all.yml
  roles/
    webserver/
      tasks/main.yml
      handlers/main.yml
      defaults/main.yml
      templates/
        nginx.conf.j2
        index.html.j2
```

`ansible.cfg`:

```
[defaults]
inventory = inventory.yml
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml
```

Gunakan role `webserver` dari Section 8.

10.2 Project 2: Provision Ubuntu Server dari Nol

`setup-server.yml`:

```

---
- name: Provision Ubuntu Server
  hosts: all
  become: yes
  vars:
    admin_user: "deploy"
    ssh_public_key: "ssh-rsa AAAAB3NzaC1yc2E..."

  tasks:
    - name: Update all packages
      ansible.builtin.apt:
        upgrade: dist
        update_cache: yes

    - name: Install essential packages
      ansible.builtin.apt:
        name:
          - curl
          - wget
          - git
          - vim
          - htop
          - net-tools
          - ufw
          - fail2ban
          - unattended-upgrades
        state: present

    - name: Create admin user
      ansible.builtin.user:
        name: "{{ admin_user }}"
        groups: sudo
        shell: /bin/bash
        state: present
        create_home: yes

    - name: Add SSH public key
      ansible.builtin.authorized_key:
        user: "{{ admin_user }}"
        key: "{{ ssh_public_key }}"
        state: present

    - name: Disable root SSH login
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^PermitRootLogin'
        line: 'PermitRootLogin no'
      notify: Restart SSH

    - name: Disable password authentication
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^PasswordAuthentication'
        line: 'PasswordAuthentication no'
      notify: Restart SSH

    - name: Setup firewall
      ansible.builtin.ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
        loop: ['22', '80', '443']

```

```
- name: Enable UFW
  ansible.builtin.ufw:
    state: enabled
    policy: deny

- name: Enable unattended security updates
  ansible.builtin.copy:
    content: |
      APT::Periodic::Update-Package-Lists "1";
      APT::Periodic::Unattended-Upgrade "1";
    dest: /etc/apt/apt.conf.d/20auto-upgrades

- name: Set timezone to UTC
  ansible.builtin.timezone:
    name: UTC

handlers:
- name: Restart SSH
  ansible.builtin.service:
    name: sshd
    state: restarted
```

10.3 Project 3: Deploy Docker + Container App

deploy-docker.yml:


```

---
- name: Deploy Docker + App
  hosts: all
  become: yes
  vars:
    app_port: 3000

  tasks:
    - name: Install Docker dependencies
      ansible.builtin.apt:
        name:
          - apt-transport-https
          - ca-certificates
          - curl
          - gnupg
          - lsb-release
        state: present

    - name: Add Docker GPG key
      ansible.builtin.apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
        state: present

    - name: Add Docker repository
      ansible.builtin.apt_repository:
        repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_rel...
        state: present

    - name: Install Docker
      ansible.builtin.apt:
        name:
          - docker-ce
          - docker-ce-cli
          - containerd.io
        state: present

    - name: Ensure Docker is running
      ansible.builtin.service:
        name: docker
        state: started
        enabled: yes

    - name: Add user to docker group
      ansible.builtin.user:
        name: "{{ ansible_user }}"
        groups: docker
        append: yes

    - name: Install docker-compose plugin
      ansible.builtin.apt:
        name: docker-compose-plugin
        state: present

    - name: Run sample app container
      ansible.builtin.docker_container:
        name: myapp
        image: nginx:alpine
        state: started
        restart_policy: always
        ports:
          - "{{ app_port }}:80"

```

11. Cheatsheet & Referensi Cepat

11.1 Command Dasar

```
# Ping semua hosts
ansible all -i inventory.yml -m ping

# Syntax check playbook
ansible-playbook playbook.yml --syntax-check

# Dry-run dengan diff
ansible-playbook playbook.yml --check --diff

# Jalankan dengan tag spesifik
ansible-playbook playbook.yml --tags deploy

# Skip tag tertentu
ansible-playbook playbook.yml --skip-tags critical

# Limit ke host tertentu
ansible-playbook playbook.yml --limit web1

# Override variable dari command line
ansible-playbook playbook.yml -e "app_version=2.0"

# Jalankan step-by-step
ansible-playbook playbook.yml --step

# List semua hosts di inventory
ansible-inventory -i inventory.yml --list

# List tasks tanpa menjalankan
ansible-playbook playbook.yml --list-tasks

# List tags yang tersedia
ansible-playbook playbook.yml --list-tags
```

11.2 Module Paling Sering Dipakai

| Module | Kegunaan | Contoh |

---|---|---

| ansible.builtin.ping | Cek konektivitas | `ansible all -m ping` |

| ansible.builtin.command | Jalankan command (no shell) | `-a "uptime"` |

| ansible.builtin.shell | Jalankan command (with shell) | `-a "ps aux | grep nginx"` |

| ansible.builtin.copy | Copy file | `src=/local/file dest=/remote/file` |

| ansible.builtin.template | Copy + render Jinja2 | `src=config.j2 dest=/etc/app.conf` |

| ansible.builtin.file | Manage file/dir/symlink | `path=/tmp/dir state=directory` |

| ansible.builtin.apt | Package manager (Debian) | `name=nginx state=present` |

| ansible.builtin.yum | Package manager (RHEL) | `name=nginx state=present` |

```
| ansible.builtin.service | Manage service | name=nginx state=started |
| ansible.builtin.user | Manage user | name=alice state=present |
| ansible.builtin.group | Manage group | name=developers state=present |
| ansible.builtin.lineinfile | Edit satu baris di file | regexp=... line=... |
| ansible.builtin.stat | Cek file info | path=/etc/hosts -> register |
| ansible.builtin.debug | Print debug message | msg="Hello {{ var }}" |
| ansible.builtin.set_fact | Set variable dinamis | key=value |
| ansible.builtin.wait_for | Tunggu port/file/condition | port=80 state=started |
| ansible.builtin.get_url | Download file | url=... dest=/tmp/file |
| ansible.builtin.uri | HTTP request | url=http://localhost status_code=200 |
| ansible.builtin.git | Git operations | repo=... dest=/path |
| ansible.builtin.cron | Manage cron jobs | name="backup" job="/script.sh" |
```

11.3 Jinja2 Filter Berguna

```
# Default value jika variable tidak terdefinisi
{{ some_var | default("fallback") }}
```

```
# Convert ke lowercase / uppercase
{{ name | lower }}
{{ name | upper }}
```

```
# Boolean operations
{{ deploy_ssl | bool }}
```

```
# List operations
{{ my_list | length }}
{{ my_list | first }}
{{ my_list | last }}
{{ my_list | join(", ") }}
```

```
# JSON
{{ my_dict | to_json }}
{{ my_dict | to_nice_json }}
```

```
# Type conversion
{{ "123" | int }}
{{ my_var | string }}
```

```
# Path manipulation
{{ path | basename }}
{{ path | dirname }}
```

```
# Regex
{{ text | regex_replace('old', 'new') }}
{{ text | regex_search('pattern') }}
```

```
# Ternary
{{ (status == "ok") | ternary("yes", "no") }}
```

11.4 ansible.cfg Konfigurasi Umum

```
[defaults]
inventory = inventory.yml
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml
gathering = smart
fact_caching_timeout = 3600
roles_path = roles
ansible_managed = "Managed by Ansible - DO NOT EDIT MANUALLY"

[ssh_connection]
pipelining = True
control_path = /tmp/ansible-%%h-%%p-%%r
```

11.5 Error Umum & Solusi

| Error | Penyebab | Solusi |

---|---|---

| UNREACHABLE | SSH gagal | Cek IP, user, port, firewall, SSH key |

| Permission denied | Tidak ada sudo | Tambah -b / become: yes |

| No such file | File tidak ditemukan | Cek path, gunakan absolute path |

| apt lock | Ada proses apt lain | Tunggu, atau kill proses apt |

| syntax error | YAML tidak valid | Cek indentasi (harus 2 spasi) |

| undefined variable | Variable tidak diset | Cek scope, gunakan | default() |

| changed=0 | Idempotent, tidak ada yang berubah | Normal. Ansible tidak melakukan apa-apa karena state sudah sesuai |

| failed: [...] msg: ... | Task gagal | Baca pesan error di msg field |

11.6 Learning Path Lanjutan

Setelah menguasai materi di guide ini, lanjutkan ke:

- Ansible Galaxy — Download role dari community (ansible-galaxy install)
- Ansible Collections — Module terorganisir untuk cloud vendor (AWS, GCP, Azure)
- AWX / Ansible Tower — Web UI + RBAC + scheduling untuk Ansible
- Ansible for Kubernetes — Module k8s untuk manage K8s resources
- Molecule — Testing framework untuk Ansible roles
- Ansible Vault + CI/CD — Integrasi vault dengan GitHub Actions / GitLab CI

Selamat belajar! Guide ini dirancang untuk dipraktekkan — buka terminal, jalankan contoh kode, bereksperimen.